

EventHub — College Event Discovery & Management Platform

Every College Event. One Place. *Discover. Register. Schedule. Never Miss an Event.*

Cover / Project Information

Field	Details
Project Name	EventHub — College Event Discovery & Management Platform
Tagline	Every College Event. One Place.
Secondary Tagline	Discover. Register. Schedule. Never Miss an Event.
Project Type	Web-Based Event Discovery and Management Platform
Domain	College Event Management / Education Technology / Community Platform
Frontend	React.js (with Vite)
Backend	Node.js + Express.js (<i>planned / integration phase</i>)
Database	MongoDB Atlas (<i>planned / integration phase</i>)
Authentication	JWT + bcryptjs (<i>planned / integration phase</i>)
Current Version	Interactive prototype / demo using mock data and a service abstraction layer
Suitable For	College project submission, hackathon documentation, technical review, project demonstration, GitHub repository documentation, final presentation

Project Team / Student Details: [Insert Team Members, Register Numbers, Guide Name, Department, Academic Year]

Documentation Conventions

Throughout this document, two labels are used consistently so that the reader can always distinguish what exists today from what is intended:

Label	Meaning
● Current Prototype	Behaviour demonstrated in the present interactive demo, which uses mock data and a service layer.
● Future Production	Functionality planned for the real backend and database integration phase. It is not presently implemented.

Important note. The current version of EventHub is an interactive prototype. It demonstrates the product experience for Students, Hosts and Administrators using mock data. It is designed so that the mock services can later be replaced by real API calls to a Node.js + Express.js backend connected to MongoDB Atlas. Any behaviour described as "demonstration-level" (for example, authentication, notifications, QR, and the EventHub AI assistant) should be understood as prototype behaviour, not production behaviour.

Table of Contents

1	Introduction
2	Problem Statement
3	Objectives
4	Scenario-Based Case Study
5	System Requirements
6	System Architecture
7	Technical Architecture
8	Database Design
9	ER Diagram
10	Key Features
11	Event Status System
12	Roles and Responsibilities
13	User Flow
14	Complete Event Lifecycle
15	Schedule Conflict Detection
16	MVC Architecture
17	Project Structure
18	Backend Development
19	Authentication and Authorization
20	Database Development
21	Frontend Development
22	Screen / Page Documentation
23	Demo Data
24	Testing
25	Security Considerations
26	Advantages
27	Limitations
28	Scalability and Future Enhancements
29	Conclusion
30	References / Project Links

1. Introduction

EventHub is a centralized, web-based college event discovery and management platform designed to simplify the complete lifecycle of college events. It brings event discovery, registration, scheduling, notifications, participation, attendance, feedback, analytics and administration into a single place.

1.1 The Current Situation in Colleges

In a typical college, events such as hackathons, workshops, symposiums, technical and cultural events, sports meets, seminars, club activities and career programs are announced through many separate channels:

- WhatsApp groups
- Department groups
- Social media
- Email
- Posters
- Separate registration forms
- Club announcements
- Other communication channels

No single channel contains all the information. A student must monitor many of them, and an announcement posted in a group the student did not see is effectively lost.

1.2 Problems Faced by Students

This fragmentation creates the following difficulties for students:

- **Missed announcements** — important events are noticed too late or never noticed.
- **Forgotten registrations** — registration deadlines pass unnoticed.
- **Difficulty finding relevant events** — there is no single place to search or filter by interest.
- **Difficulty tracking multiple events** — registrations are spread across different forms and platforms.
- **Schedule conflicts** — overlapping events are noticed only at the last moment.
- **Lack of timely reminders** — there is no unified reminder mechanism.
- **Unclear event state** — it is hard to tell whether an event is upcoming, live, completed, cancelled or full.

1.3 Problems Faced by Organizers

Organizers face a parallel set of problems:

- **Manual registration management** through separate forms and spreadsheets.
- **Participant list management** across multiple sources.
- **Attendance tracking** done on paper or informally.
- **Notifications** that must be sent manually through several channels.
- **Feedback collection** that is disconnected from registration.
- **Event analytics** that are rarely available in any structured form.
- **Communication with participants** that depends on informal groups.

1.4 The Need for a Centralized Platform

A centralized platform removes the need to monitor many channels. It gives students one place to discover and track events, gives organizers one place to manage them, and gives administrators one place to verify and monitor them.

1.5 EventHub as the Solution

EventHub connects three types of users:

User Type	Role in the Platform
Students	Discover events, register, manage their schedule, receive notifications, attend and give feedback.
Hosts / Organizers	Create and manage events, manage participants and attendance, monitor feedback and analytics.
Administrators	Verify and approve events, manage users, roles and categories, and monitor the platform.

The complete EventHub lifecycle is:

Discover → Register → Schedule → GetNotified → Attend → Feedback

- **Current Prototype:** The present version demonstrates these functionalities using mock data and a service abstraction layer.
- **Future Production:** The architecture is designed for later integration with Node.js, Express.js, MongoDB Atlas, JWT authentication and real-time services.

2. Problem Statement

In most colleges, information about events is highly fragmented. Announcements, posters, registration forms and updates are distributed across multiple communication channels such as WhatsApp groups, department groups, social media, email and club announcements. There is no single, authoritative source of event information.

This fragmentation leads to the following problems:

- 1 **Fragmented event information.** Details of a single event may be spread across a poster, a chat message and a separate form, and may differ between them.
- 2 **Multiple communication channels.** Students must monitor many channels to stay informed, and no channel is guaranteed to reach everyone.
- 3 **Missed opportunities.** Students miss workshops, competitions and career programs because the information did not reach them in time.
- 4 **Manual registration processes.** Registrations are collected through separate forms, requiring organizers to consolidate and verify data by hand.
- 5 **Poor schedule management.** Students have no consolidated view of the events they have committed to, and overlapping events go unnoticed.
- 6 **Lack of centralized verification.** Students cannot easily determine whether an event announcement is genuine or officially approved.
- 7 **Difficult event administration.** Organizers and administrators lack tools to manage capacity, participants, attendance and communication in one place.
- 8 **Lack of unified analytics.** Registration, attendance and feedback data are not collected in a common format, so the effectiveness of events cannot be measured.

Therefore, there is a clear need for a **centralized, intelligent and scalable event management platform** that connects students, event organizers and administrators throughout the complete event

3. Objectives

The objectives of EventHub are as follows.

No.	Objective	Description
1	Centralize college event information	Provide one platform where all college events are listed.
2	Enable relevant event discovery	Allow students to search, filter and browse events by category, college, date, type and status.
3	Support inter-college event discovery	Allow events from multiple colleges to be discovered in one place.
4	Provide verified event information	Ensure events pass administrator verification before being treated as verified.
5	Simplify event registration	Allow students to register directly from the event page.
6	Provide personal event schedules	Give each student a schedule view of their registered events.
7	Detect schedule conflicts	Identify overlapping registered events automatically.
8	Provide reminders and notifications	Keep students informed about registered events and updates.
9	Support LIVE NOW events	Clearly identify events that are currently running.
10	Support capacity and waitlists	Track registrations against capacity and support a waitlist when an event is full.
11	Help organizers create and manage events	Provide hosts with tools to create, edit, submit and cancel events.
12	Support participant management	Let hosts view and manage registered participants.
13	Support attendance tracking	Let hosts record attendance for events.
14	Collect event feedback	Allow participants to submit ratings and comments.
15	Provide event analytics	Present registration, attendance and feedback statistics to hosts and administrators.
16	Allow administrators to verify and manage events	Provide approval, rejection and monitoring tools.
17	Provide role-based access	Restrict features according to Student, Host and Admin roles.
18	Provide an EventHub AI assistant	Offer an assistant that helps users find event information.
19	Create a scalable architecture	Design the system so it can be integrated with MongoDB Atlas and real authentication.

4. Scenario-Based Case Study

This case study illustrates how EventHub is used by each type of user across the complete event lifecycle. The persons and events are fictional and used for demonstration.

4.1 Background

Sivaperumal is a college student who frequently participates in hackathons, workshops, symposiums, technical events, cultural events and career programs.

At present, he receives event information through several different WhatsApp groups, department announcements, posters and separate registration forms. He often learns about events late, forgets to register, and occasionally finds that two events he wanted to attend take place at the same time.

He discovers EventHub and decides to use it as his single event platform.

4.2 The Student Journey

Step 1 – User Registration and Authentication. Sivaperumal creates an EventHub account by providing his name, email, college, department and interests, and then logs in. (● *In the prototype, authentication is demonstration-level.* ● *In production, passwords will be hashed with bcryptjs and sessions will use JWT.*)

Step 2 – Event Discovery. After logging in he lands on his home screen and opens **Discover Events**, where events from his own and other colleges are listed in one place.

Step 3 – Search and Filtering. He searches for "hackathon" and then narrows the results using filters such as category, college, date, event type and status.

Step 4 – Viewing Event Details. He opens the *Inter-College Hackathon* and reviews its description, date, time, venue, organizer, capacity, eligibility, registration deadline, rules and speakers. A verification indicator shows whether the event has been approved by an administrator.

Step 5 – Event Registration. He registers for the event directly from the event page. The event is added to his **My Events** section, and the registered count for the event is updated. Had the event been full, he would have been offered the waitlist.

Step 6 – Schedule Management. The registered event also appears in **My Schedule**, where he can see all the events he has committed to in date and time order.

Step 7 – Schedule Conflict Detection. Sivaperumal then tries to register for a workshop that overlaps with the hackathon. EventHub compares the timings of the events and warns him that a schedule conflict has been detected, so he can make an informed decision.

Step 8 – Notifications and Reminders. He receives notifications and reminders about his registered events and important updates, and can review them in the **Notifications** section. (● *Prototype-level in the current version.*)

Step 9 – LIVE NOW Events. When an event is in progress, EventHub marks it with a **LIVE NOW** status so that students can immediately see what is happening at that moment.

Step 10 – Event Sharing and QR. Sivaperumal shares the event with friends and can display an event QR. (● *QR functionality may be demonstration-level in the current prototype.*)

Step 11 – Event Attendance. He attends the event and his attendance is recorded by the host, so that it is associated with his registration.

Step 12 – Feedback. After the event, he submits a rating and comment through the feedback feature.

Step 13 – Achievements and Certificates. His participation is recorded as an achievement or certificate entry in his profile.

Step 14 – EventHub AI Assistant. When he is unsure which event to pick, he asks the **EventHub AI** assistant for help finding events. (● *The assistant currently uses demo / rule-based logic.* ● *Real AI integration is a future enhancement.*)

4.3 The Organizer Experience

A host from a department wants to organize a workshop.

- 1 **Organizer Login.** The host logs in with a Host account.
- 2 **Organizer Dashboard.** The Host Dashboard shows the host's events, registrations, attendance and analytics.
- 3 **Create Event.** The host fills in the event details: title, description, category, date, time, venue, event type, capacity, registration deadline, eligibility, rules, speakers and schedule.
- 4 **Submit Event.** The host submits the event for approval; its verification status becomes *Pending*.

- 5 **Admin Approval.** An administrator reviews the event and approves or rejects it.
- 6 **Manage Event.** Once approved and published, the host can edit or cancel the event and monitor capacity.
- 7 **Participant Management.** The host views the list of registered participants.
- 8 **Attendance.** On the event day the host records attendance.
- 9 **Feedback.** After the event, the host reviews the ratings and comments submitted by participants.
- 10 **Analytics.** The host views statistics on registrations, attendance and feedback to judge the success of the event.

4.4 The Administrator Experience

- 1 **Admin Login.** The administrator logs in with an Admin account.
- 2 **Admin Dashboard.** The dashboard summarizes users, events, pending approvals and platform activity.
- 3 **Event Approval.** Pending events are reviewed and then approved or rejected.
- 4 **User Management.** The administrator manages user accounts and roles.
- 5 **Category Management.** Event categories are created and maintained.
- 6 **Event Management.** The administrator can view and manage all events on the platform.
- 7 **Analytics.** Platform-level statistics are reviewed.
- 8 **Platform Monitoring.** The administrator monitors overall activity to maintain platform integrity.

4.5 Outcome

Before EventHub, Sivaperumal depended on scattered messages, and organizers depended on manual forms and lists. With EventHub, events are discovered in one place, registrations and schedules are tracked automatically, conflicts are flagged, event status is always visible, organizers manage participants and attendance centrally, and administrators verify and monitor the ecosystem. EventHub therefore improves every stage of the event lifecycle: **discovery, registration, scheduling, notification, participation, attendance, feedback and analysis.**

5. System Requirements

5.A Software Requirements

Software	Purpose	Current Prototype	Future Production
Windows 10/11, macOS or Linux	Development environment	✓	✓
Node.js	JavaScript runtime (also runs the Vite toolchain)	✓	✓
npm	Package management	✓	✓
JavaScript	Programming language	✓	✓
React.js	Frontend development	✓	✓
Vite	Frontend build tool and dev server	✓	✓
Git	Version control	✓	✓
GitHub	Source-code repository	✓	✓
Visual Studio Code	Code editor	✓	✓
Google Chrome / Firefox	Application testing	✓	✓
Express.js	Backend REST API framework	—	Planned
MongoDB Atlas	Cloud-hosted production database	—	Planned
Mongoose	MongoDB object modeling	—	Planned
JWT and bcryptjs	Authentication and password hashing	—	Planned
Postman	REST API testing	—	Planned

Current Prototype. The prototype relies on the React frontend, reusable components, the service abstraction layer and mock data. It does not require a database server or a backend API.

Future Production. The production version will integrate the stack: [React](#) → [REST API](#) → [Express on Node.js](#) → [Mongoose](#) → [MongoDB Atlas](#) .

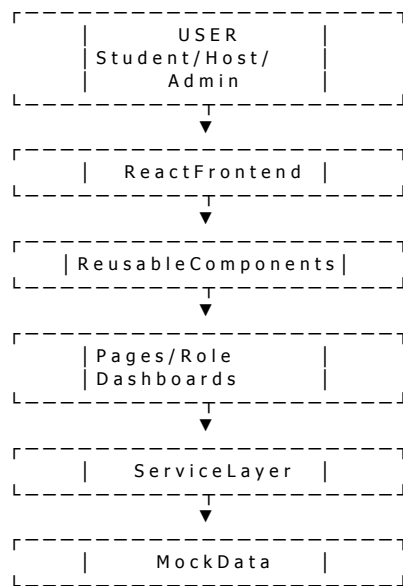
Note on MongoDB. EventHub uses [MongoDB Atlas](#), a cloud-hosted database. MongoDB is therefore **not** installed locally; the backend connects to Atlas using a connection string stored in an environment variable.

5.B Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i5 / AMD Ryzen 5	Intel Core i5/i7 or AMD Ryzen 5/7, or better
RAM	8 GB	16 GB
Storage	At least 5 GB free	SSD storage with 5 GB or more free
Display	1366 × 768 or higher	Full HD (1920 × 1080)
Network	Stable internet connection	Stable broadband connection

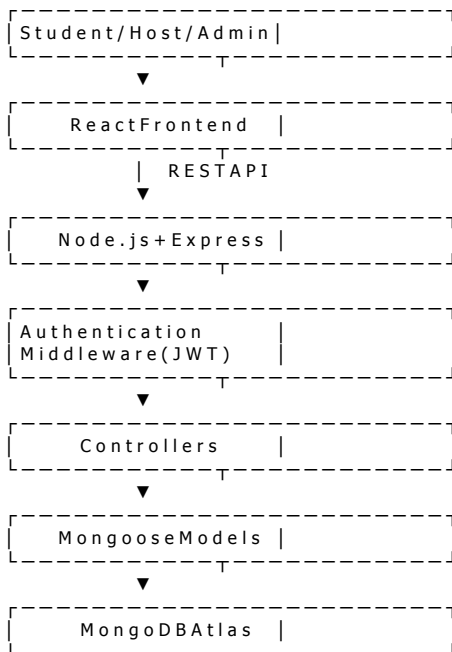
6. System Architecture

6.1 Current Prototype Architecture •



[Insert Diagram — Current Prototype Architecture]

6.2 Future Production Architecture •



[Insert Diagram — Future Production Architecture]

6.3 Explanation of Components

- **Frontend.** The React frontend handles user interaction and presentation for all three roles.
- **Service layer.** The service layer abstracts data operations so that mock services can later be replaced with real API calls, without redesigning the user interface.
- **Backend (planned).** The Node.js + Express backend will handle authentication, authorization, event management, registration, notifications, attendance, feedback and analytics.
- **Database (planned).** MongoDB Atlas will store persistent application data.

7. Technical Architecture

EventHub follows a layered architecture in which each layer has one responsibility.

7.1 Layers

Layer	Responsibility	Status
Presentation Layer	React components, pages, layouts and role dashboards that render the interface and capture user actions.	● Prototype
Service Layer	A single point of access for data operations, separating the UI from the data source.	● Prototype (mock) → ● API calls
Backend Layer	Express server, authentication middleware, controllers and routes.	● Planned
Database Layer	Mongoose models and MongoDB Atlas collections for persistent data.	● Planned

7.2 Production Request-Response Flow ●

```
Student
↓
EventHubReactInterface
↓
APIRequest
↓
ExpressServer
↓
AuthenticationMiddleware
↓ Controller
↓
MongooseModel
↓
MongoDBAtlas
↓ Response
↓ ReactUI
```

- 1 **Student / EventHub React Interface.** The user performs an action such as registering for an event.
- 2 **API Request.** The service layer sends an HTTP request to the REST API, including the user's token.
- 3 **Express Server.** Receives the request and routes it to the matching route.
- 4 **Authentication Middleware.** Verifies the JWT and checks that the user's role is permitted to perform the action.
- 5 **Controller.** Applies the business logic, for example checking capacity and duplicate registrations.
- 6 **Mongoose Model.** Validates and reads or writes data using the defined schema.
- 7 **MongoDB Atlas.** Stores or retrieves the documents.
- 8 **Response.** The result is returned as JSON.
- 9 **React UI.** The interface updates and shows a toast message, updated list or error state.

7.3 Current Demo Flow Using Mock Data ●

```

UseractioninReactUI
↓
Page/Component
↓
ServiceLayerfunction
↓
MockData(in-memory/localsamplerrecords)
↓
Resultreturnedtotheservicelayer
↓
UIupdates

```

In the demo, no HTTP request is sent to a server and no database is queried. Because every page calls the service layer rather than reading mock data directly, moving to production requires changing the service implementations, not the pages.

8. Database Design

- **Future Production.** EventHub is designed to use **MongoDB Atlas** as its production database. The prototype does not currently connect to a database; it uses mock data shaped like the documents below.

The database contains seven collections:

No.	Collection	Purpose
1	users	Accounts for students, hosts and administrators.
2	events	Event details, status and verification.
3	categories	Event categories managed by administrators.
4	registrations	Links between users and events they registered for.
5	notifications	Messages and reminders sent to users.
6	feedback	Ratings and comments from participants.
7	attendance	Attendance records per user and event.

8.1 Users

Field	Description
_id	Unique identifier (ObjectId)
name	Full name
email	Email address (used for login)
password	Password, stored only as a bcryptjs hash
college	College of the user
department	Department of the user
role	Student, Host or Admin
interests	User's interests, used for recommendations
createdAt	Account creation time

8.2 Events

Field	Description
<code>_id</code>	Unique identifier
<code>title</code>	Event title
<code>description</code>	Event description
<code>college</code>	Hosting college
<code>category</code>	Reference to a category
<code>organizer</code>	Organizer name / organization
<code>createdBy</code>	Reference to the host user who created the event
<code>date</code>	Event date
<code>startTime</code>	Start time
<code>endTime</code>	End time
<code>venue</code>	Location
<code>eventType</code>	Type of event (for example online or offline)
<code>capacity</code>	Maximum number of participants
<code>registeredCount</code>	Current number of registrations
<code>registrationDeadline</code>	Last date for registration
<code>banner</code>	Banner image
<code>eligibility</code>	Who may participate
<code>rules</code>	Event rules
<code>speakers</code>	Speakers / guests
<code>schedule</code>	Internal schedule of the event
<code>liveMeetingLink</code>	Link for online participation
<code>status</code>	Upcoming, LIVE NOW, Completed, Cancelled or Full
<code>verificationStatus</code>	Pending, Approved or Rejected
<code>isExternal</code>	Indicates an event from another college
<code>timestamps</code>	Created and updated times

8.3 Categories

Field	Description
<code>_id</code>	Unique identifier
<code>name</code>	Category name
<code>description</code>	Category description
<code>status</code>	Whether the category is active

8.4 Registrations

Field	Description
<code>_id</code>	Unique identifier
<code>user</code>	Reference to the registered user
<code>event</code>	Reference to the event
<code>registeredAt</code>	Registration time
<code>status</code>	Registration status
<code>attendanceStatus</code>	Attendance status of this registration

8.5 Notifications

Field	Description
<code>_id</code>	Unique identifier
<code>user</code>	Recipient user
<code>event</code>	Related event
<code>title</code>	Notification title
<code>message</code>	Notification message
<code>type</code>	Notification type
<code>read</code>	Whether the notification has been read
<code>createdAt</code>	Creation time

8.6 Feedback

Field	Description
<code>_id</code>	Unique identifier
<code>user</code>	User who submitted the feedback
<code>event</code>	Event being reviewed
<code>rating</code>	Numeric rating
<code>comment</code>	Written comment
<code>createdAt</code>	Submission time

8.7 Attendance

Field	Description
<code>_id</code>	Unique identifier
<code>user</code>	Attending user
<code>event</code>	Event attended
<code>attendanceStatus</code>	Attendance status
<code>markedAt</code>	Time attendance was marked

8.8 Relationships Between Collections

- A **User** (host) creates many **Events** (`events.createdBy` → `users._id`).
- A **Category** contains many **Events** (`events.category` → `categories._id`).
- A **User** (student) has many **Registrations**, and an **Event** has many **Registrations**.
- A **User** receives many **Notifications**, which may relate to an **Event**.
- A **User** gives many **Feedback** entries, and an **Event** receives many.
- A **User** has many **Attendance** records, and an **Event** has many.

9. ER Diagram

MongoDB is a document-oriented (NoSQL) database, so the diagram below is a **logical ER diagram**. In implementation, relationships are represented by **ObjectId references** between documents rather than SQL foreign keys. **PK** denotes the primary identifier (`_id`) and **FK** denotes a reference field.

9.1 Entities and Relationships

Relationship	Meaning	Cardinality
USER → creates → EVENT	A host creates events	1 : N
USER → registers for → EVENT	Through REGISTRATION	M : N
EVENT → belongs to → CATEGORY	Each event has one category	N : 1
USER → receives → NOTIFICATION	A user receives notifications	1 : N
USER → gives → FEEDBACK	A user submits feedback	1 : N
USER → has → ATTENDANCE	A user has attendance records	1 : N
EVENT → has → REGISTRATIONS	An event has many registrations	1 : N
EVENT → has → FEEDBACK	An event receives feedback	1 : N
EVENT → has → ATTENDANCE	An event has attendance records	1 : N

9.2 Diagram (Text Form)

```

CATEGORY(PK_id,name,description,status)
    |
    | 1
    | N
    |
EVENT(PK_id,title,...,categoryFK,createdByFK,status,verificationStatus)
    |           |           |           |           |
    | 1         | 1         | 1         | 1         |
    | N         | N         | N         | N         |
    |           |           |           |           |
REGISTRATION ATTENDANCE FEEDBACK NOTIFICATION
(userFK,      (userFK,    (userFK,    (userFK,
 eventFK)     eventFK)    eventFK)    eventFK)
    |           |           |           |
    | N         | N         | N         | N
    | 1         | 1         | 1         | 1
    |           |           |           |
USER(PK_id,name,email,password,college,department,role,interests,createdAt)
    |
    | 1:N --> EVENT(createdBy) [hostcreatesevents]
  
```

[Insert Diagram — EventHub ER Diagram (generate from the description above using [draw.io](#), [Lucidchart](#) or [dbdiagram.io](#))]

9.3 Identifiers and References

- Every collection uses `_id` (an ObjectId) as its primary identifier.
- `events.createdBy` references `users._id`.
- `events.category` references `categories._id`.
- `registrations.user`, `notifications.user`, `feedback.user` and `attendance.user` reference `users._id`.
- `registrations.event`, `notifications.event`, `feedback.event` and `attendance.event` reference `events._id`.

10. Key Features

Status note. In the current prototype these features are demonstrated using mock data. Features that depend on backend services (real authentication, persistent storage, email or push delivery, real AI, real QR scanning) are prototype-level and are completed in the production phase. **Before submission, the team should confirm each row against the actual build.**

10.1 Student Features

No.	Feature	Description
1	User Registration	Create an account with name, email, college, department and interests.
2	Login	Sign in to access role-specific features.
3	Profile	View and maintain personal details.
4	Event Discovery	Browse events from a centralized interface.
5	Search	Find events by keyword.
6	Filters	Filter by category, college, date, event type and status.
7	Categories	Browse events by category.
8	Inter-College Events	Discover events hosted by other colleges.
9	Verified Events	Identify events approved by administrators.
10	Event Details	View description, date, time, venue, organizer, capacity, eligibility, deadline, rules and speakers.
11	Online Registration	Register directly from the event page.
12	Registration Cancellation	Cancel an existing registration.
13	Capacity Management	Capacity and registered count are tracked per event.
14	Waitlist	Join a waitlist when an event is full.
15	My Events	View registered events.
16	My Schedule	View registered events by date and time.
17	Schedule Conflict Detection	Identify overlapping registered events.
18	Notifications	View event-related notifications.
19	Smart Reminders	Receive reminders for registered events.
20	LIVE NOW	See events that are currently running.
21	Event Sharing	Share events with others.
22	Event QR	Display a QR for an event.
23	Personalized Recommendations	Suggestions based on the user's interests.
24	Feedback	Submit ratings and comments.
25	Achievements	Record participation as achievements.
26	Certificates	Record participation certificates.
27	EventHub AI	Assistant that helps find event information.

10.2 Host Features

No.	Feature	Description
1	Host Dashboard	Overview of the host's events and statistics.
2	Create Event	Enter all event details in a form.
3	Edit Event	Update event information.
4	Delete / Cancel Event	Remove or cancel an event.
5	Submit for Approval	Send the event for administrator review.
6	Registration Monitoring	Track registrations.
7	Participant Management	View and manage participants.
8	Capacity Management	Monitor capacity against registrations.
9	Attendance Management	Record attendance.
10	Notifications	Send and receive event-related updates.
11	Feedback Monitoring	Review feedback and ratings.
12	Event Analytics	View event statistics.

10.3 Admin Features

No.	Feature	Description
1	Admin Dashboard	Platform overview.
2	Event Approval	Approve pending events.
3	Event Rejection	Reject events that do not meet requirements.
4	Event Management	View and manage all events.
5	User Management	Manage user accounts.
6	Role Management	Manage user roles.
7	Category Management	Create and maintain categories.
8	Platform Analytics	View platform-wide statistics.
9	Platform Monitoring	Monitor overall activity.

10.4 Common Features

- Responsive UI
- Search
- Notifications
- Toast messages
- Loading states
- Empty states
- Error states
- Accessibility considerations
- Quick actions
- Global navigation

11. Event Status System

EventHub uses two independent status systems: an **event status** describing where the event is in time and capacity, and a **verification status** describing whether it has been approved.

11.1 Event Status

Status	Meaning	Why It Exists
Upcoming	The event has not yet started.	Lets students know they can still register and plan.
LIVE NOW	The event is currently happening.	Helps students immediately find and join running events.
Completed	The event has finished.	Moves the event out of active discovery and enables attendance review, feedback and analytics.
Cancelled	The event has been cancelled.	Prevents students from attending or registering for an event that will not happen.
Full	The event has reached capacity.	Stops further direct registration and enables the waitlist.

11.2 Verification Status

Status	Meaning	Why It Exists
Pending	Awaiting administrator review.	Prevents unreviewed events from being treated as verified.
Approved	Verified by an administrator.	Gives students confidence that the information is genuine.
Rejected	Not approved.	Allows administrators to block events that do not meet requirements.

12. Roles and Responsibilities

12.1 Student

- Register and log in
- Discover events
- Search and filter
- View event details
- Register for events
- Cancel registration
- Manage schedule
- Receive notifications
- Attend events
- Submit feedback
- View achievements

12.2 Host / Organizer

- Create events
- Edit events
- Submit events for approval
- Manage registrations
- Manage participants
- Track attendance

- Monitor feedback
- View analytics
- Provide accurate event information

12.3 Administrator

- Manage users
- Approve or reject events
- Manage categories
- Monitor events
- View analytics
- Maintain platform integrity

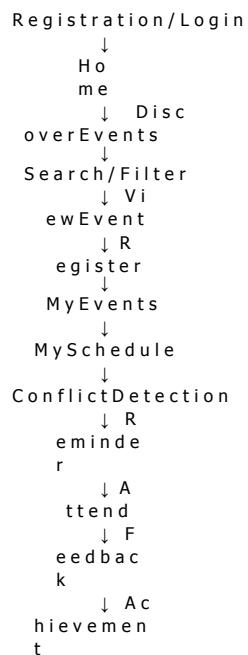
12.4 Access Summary

Capability	Student	Host	Admin
Discover and register for events	✓	✓	✓
Submit feedback	✓	—	—
Create / edit events	—	✓	✓
Manage participants and attendance	—	✓	✓
Approve / reject events	—	—	✓
Manage users and categories	—	—	✓

(Confirm the Admin's rights over event editing against the implemented prototype.)

13. User Flow

13.1 Student Flow



13.2 Host Flow



13.3 Admin Flow

Login→AdminDashboard→ReviewEvents→Approve/Reject
→ManageUsers→ManageCategories→MonitorEvents→ViewAnalytics

[Insert Diagram — Student, Host and Admin User Flows]

14. Complete Event Lifecycle

Draft→Submit→PendingApproval→AdminApproval→Published
→Upcoming→Registration→LIVENOW→Completed
→Attendance→Feedback→Analytics

Stage	Explanation
Draft	The host prepares event details; the event is not yet visible to others.
Submit	The host submits the event for review.
Pending Approval	The event waits with verification status <i>Pending</i> .
Admin Approval	An administrator approves or rejects it. Rejected events do not proceed.
Published	The approved event becomes visible in discovery.
Upcoming	The event is listed as not yet started.
Registration	Students register until the deadline or until capacity is reached; a waitlist is available when full.
LIVE NOW	The event is in progress and is highlighted.
Completed	The event has finished.
Attendance	The host records who attended.
Feedback	Participants submit ratings and comments.
Analytics	Registrations, attendance and feedback are analysed to evaluate the event.

Registration overlaps with the Upcoming stage in practice; they are listed separately to emphasize the student's action. A **Cancelled** status can be applied by the host at any stage before completion.

[Insert Diagram — Complete Event Lifecycle]

15. Schedule Conflict Detection

EventHub identifies whether two events registered by a student overlap in time.

15.1 Logic

Two events A and B overlap when each begins before the other ends:

```
EventAStart<EventBEnd  
AND  
EventBStart<EventAEnd
```

If both conditions are true, a **Schedule Conflict** is detected.

15.2 Example

```
EventA: 10:00AM-----12:00PM  
EventB:      11:00AM-----1:00PM
```

- A start (10:00) < B end (1:00 PM) → **true**
- B start (11:00) < A end (12:00) → **true**

Result: Schedule Conflict Detected. If Event B instead started at 12:00 PM, the second condition (12:00 < 12:00) would be false and there would be no conflict, because the events only touch at a boundary.

15.3 Illustrative Code Placeholder

```
//Illustrative logic only — adapt to the actual implementation
function hasConflict(a,b){
  return a.start < b.end && b.start < a.end;
}
```

Events must be compared on the same date. The check is applied when a student attempts to register, against the student's existing registered events.

16. MVC Architecture

- **Future Production.** The planned backend follows an MVC-style separation.

16.1 Model

Models define the database schemas: **User, Event, Category, Registration, Notification, Feedback, Attendance.**

16.2 Controller

Controllers contain the business logic: **Authentication Controller, User Controller, Event Controller, Registration Controller, Notification Controller, Feedback Controller, Analytics Controller.**

16.3 Routes

Routes map API endpoints to controllers: **authRoutes, userRoutes, eventRoutes, registrationRoutes, notificationRoutes, feedbackRoutes, analyticsRoutes.**

16.4 View

In EventHub the "view" is the React frontend, which consumes JSON from the API.

16.5 Advantages

Advantage	Explanation
Separation of concerns	Data, logic and routing are kept apart.
Maintainability	Changes are localized.
Scalability	New modules can be added without disturbing existing ones.
Reusability	Models and middleware are shared across controllers.
Testing	Each layer can be tested independently.
Team collaboration	Team members can work on different modules in parallel.

[Insert Diagram — MVC Architecture]

17. Project Structure

- **Planned production structure:**

```

EventHub/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── layouts/
│   │   ├── services/
│   │   ├── context/
│   │   ├── hooks/
│   │   ├── utils/
│   │   ├── App.jsx
│   │   └── main.jsx
│   └── package.json
├── server/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── seed/
│   ├── utils/
│   ├── .env
│   ├── .env.example
│   ├── server.js
│   └── package.json
├── .gitignore
└── README.md

```

Folder / File	Purpose
client/src/components	Reusable UI components.
client/src/pages	Page-level screens for each role.
client/src/layouts	Shared layouts such as navigation and dashboards.
client/src/services	Service layer; mock in the prototype, API calls in production.
client/src/context	Shared state such as the logged-in user.
client/src/hooks	Custom React hooks.
client/src/utils	Helper functions such as date handling and conflict checks.
server/config	Configuration such as the database connection.
server/controllers	Business logic for each module.
server/middleware	Authentication, authorization and error handling.
server/models	Mongoose schemas.
server/routes	API route definitions.
server/seed	Sample data scripts.
server/.env	Secret configuration; never committed.
server/.env.example	Template listing required variables without values.
server/server.js	Backend entry point.
.gitignore	Excludes <code>.env</code> , <code>node_modules</code> and similar files.
README.md	Repository overview.

The `server/` folder belongs to the planned integration phase. The prototype's actual folder layout may differ; update this section to match the repository.

18. Backend Development

- **Future Production.** None of the backend described in this section is implemented in the current prototype.

18.1 Technologies

Technology	Role
Node.js	JavaScript runtime for the server.
Express.js	Web framework for building REST APIs.
REST APIs	Standard HTTP interface between frontend and backend.
Mongoose	Defines schemas and models for MongoDB.
MongoDB Atlas	Cloud database for persistent data.
JWT	Token-based authentication.
bcryptjs	Password hashing.
Middleware	Functions that run before controllers (authentication, role checks, validation, error handling).
Controllers	Contain the business logic.
Routes	Map URLs and HTTP methods to controllers.

18.2 Planned API Endpoints (Not Yet Implemented)

Authentication

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Log in
GET	/api/auth/me	Get current user
POST	/api/auth/logout	Log out

Events

Method	Endpoint	Purpose
GET	/api/events	List events
GET	/api/events/:id	Event details
POST	/api/events	Create event
PUT	/api/events/:id	Update event
DELETE	/api/events/:id	Delete event
PATCH	/api/events/:id/status	Update event status
PATCH	/api/events/:id/approve	Approve event (Admin)
PATCH	/api/events/:id/reject	Reject event (Admin)
GET	/api/events/live	List LIVE NOW events
GET	/api/events/upcoming	List upcoming events
GET	/api/events/my-events	Events created by the current host

Registrations

Method	Endpoint	Purpose
POST	/api/registrations/:eventId	Register for an event
GET	/api/registrations/my	Current user's registrations
DELETE	/api/registrations/:eventId	Cancel registration
GET	/api/registrations/event/:eventId	Registrations of an event (Host/Admin)
PATCH	/api/registrations/:id/attendance	Update attendance

Notifications

Method	Endpoint	Purpose
GET	/api/notifications	List notifications
PATCH	/api/notifications/:id/read	Mark one as read
PATCH	/api/notifications/read-all	Mark all as read

Health

Method	Endpoint	Purpose
GET	/api/health	Server health check

19. Authentication and Authorization

19.1 Current Prototype •

Authentication in the prototype is demonstration-level. It exists to show role-based navigation and is not a secure production mechanism.

19.2 Planned Production Authentication •

Registration

User enters details → Validation → Password hashing (bcryptjs) → Store user in MongoDB

Login

Email + Password → Validate credentials → Generate JWT → Authenticated session

19.3 Authorization •

Role	Access
Student	Student features
Host	Host features
Admin	Admin features

The authentication middleware verifies the JWT on each protected request, and a role check ensures the user may perform the requested action.

19.4 Rules

- Passwords must **never** be stored as plain text; only bcryptjs hashes are stored.
- The JWT secret and MongoDB credentials must be stored in **environment variables**, not in source code.

20. Database Development

- **Future Production.** The following steps describe the MongoDB Atlas integration plan.

- 1 **Create a MongoDB Atlas cluster.**
- 2 **Create a database** (for example, **eventhub**).
- 3 **Configure a database user** with a strong password and limited privileges.
- 4 **Configure network access** (allow only required IP addresses).
- 5 **Add MONGODB_URI** to the server's **.env** file.
- 6 **Install Mongoose** (**npm install mongoose**).
- 7 **Create schemas** for the seven collections.

- 8 **Create models** from the schemas.
- 9 **Connect the backend** to Atlas from [server/config](#).
- 10 **Test database operations** (create, read, update, delete) using Postman and seed data.

20.1 Environment Variables

```
PORT=5000
MONGODB_URI=<your-mongodb-atlas-connection-string>JWT_SECRET=<your-secret-key>CLIENT_URL=<your-frontend-url>
```

Do not expose real credentials in documentation or in GitHub. Commit only `.env.example` with placeholder values and list `.env` in `.gitignore`.

20.2 Collection Structure

```
eventhub
├--users
├--events
├--categories
├--registrations
├--notifications
├--feedback
└--attendance
```

21. Frontend Development

- **Current Prototype.** The frontend is built with **React.js** and **Vite**.

21.1 Frontend Sections

Group	Sections
Student	Landing/Home, Discover, Event Details, My Events, My Schedule, Notifications, Profile, Achievements, EventHub AI
Host	Host Dashboard, Create Event, Manage Events, Participants, Attendance, Analytics
Admin	Admin Dashboard, Approvals, Users, Categories, Analytics

21.2 Reusable Components

Reusable components keep the interface consistent and reduce duplication. Typical components for a project of this kind include event cards, filter panels, search bars, status badges, navigation bars, dashboards, forms, tables, modals, toast messages, loading indicators, empty-state and error-state views. [\[Update this list with the actual component names from the repository.\]](#)

21.3 Service Layer

Pages call service functions rather than accessing mock data directly. In production, these functions will call the REST API.

21.4 UI Quality Considerations

Responsive layout, loading states, empty states, error states, toast messages and accessibility considerations are part of the interface design.

22. Screen / Page Documentation

Screenshots are intentionally left as placeholders. Replace each with a real screenshot of the running prototype.

22.1 Landing Page

- **Purpose:** Introduce EventHub and lead users to event discovery.

- **Main UI elements:** Branding, tagline, search, featured events, upcoming events, LIVE NOW events, navigation.
- **User actions:** Search, browse featured events, log in or register.
- **Expected result:** The user understands the platform and can reach discovery or authentication.
- [\[Insert Screenshot — EventHub Landing Page\]](#)

22.2 Login

- **Purpose:** Authenticate users.
- **Main UI elements:** Email and password fields, login button, link to registration.
- **User actions:** Enter credentials and submit.
- **Expected result:** The user is directed to the dashboard for their role. Invalid input shows an error.
- [\[Insert Screenshot — Login\]](#)

22.3 Registration

- **Purpose:** Create a new account.
- **Main UI elements:** Name, email, password, college, department and interests fields.
- **User actions:** Fill in the form and submit.
- **Expected result:** An account is created; validation errors are shown for invalid input.
- [\[Insert Screenshot — Registration\]](#)

22.4 Home Dashboard

- **Purpose:** Give students a personalized starting point.
- **Main UI elements:** Recommended events, upcoming events, LIVE NOW, quick actions.
- **User actions:** Open events, use quick actions.
- **Expected result:** Quick access to relevant events and features.
- [\[Insert Screenshot — Student Dashboard\]](#)

22.5 Discover Events

- **Purpose:** Find relevant events.
- **Main UI elements:** Search bar; category, college, date, type and status filters; event cards.
- **User actions:** Search, filter, open an event.
- **Expected result:** Matching events are displayed; an empty state appears if none match.
- [\[Insert Screenshot — Discover Events\]](#)

22.6 Event Details

- **Purpose:** Show complete event information.
- **Main UI elements:** Title, banner, description, date, time, venue, organizer, capacity, eligibility, deadline, rules, speakers, verification and status badges, register button.
- **User actions:** Register, join the waitlist, share, view QR.
- **Expected result:** The student can make an informed decision.
- [\[Insert Screenshot — Event Details\]](#)

22.7 Registration (Event Registration)

- **Purpose:** Register for an event.
- **Main UI elements:** Register / cancel button, capacity indicator, waitlist option, confirmation message.
- **User actions:** Register or cancel.
- **Expected result:** The event is added to or removed from My Events and the registered count updates; a conflict warning appears if timings overlap.
- [\[Insert Screenshot — Event Registration\]](#)

22.8 My Events

- **Purpose:** Track registered events.
- **Main UI elements:** List of upcoming and past registered events.
- **User actions:** Open, cancel, give feedback.
- **Expected result:** The student sees all registrations in one place.
- [\[Insert Screenshot — My Events\]](#)

22.9 My Schedule

- **Purpose:** Show the student's personal schedule.
- **Main UI elements:** Date/time ordered schedule, conflict indicators.
- **User actions:** View schedule, open events.
- **Expected result:** A clear schedule with overlapping events flagged.
- [\[Insert Screenshot — My Schedule\]](#)

22.10 Notifications

- **Purpose:** Show updates and reminders.
- **Main UI elements:** Notification list, read/unread state, mark-as-read actions.
- **User actions:** Read and mark notifications.
- **Expected result:** Notifications update their read state.
- [\[Insert Screenshot — Notifications\]](#)

22.11 LIVE NOW

- **Purpose:** Highlight events in progress.
- **Main UI elements:** LIVE NOW badges and listings, meeting link where applicable.
- **User actions:** Open live events.
- **Expected result:** Students quickly find running events.
- [\[Insert Screenshot — LIVE NOW\]](#)

22.12 Profile

- **Purpose:** Display and manage personal details.
- **Main UI elements:** Name, college, department, interests.

- **User actions:** View or update profile.
- **Expected result:** Profile information is displayed.
- [\[Insert Screenshot — Profile\]](#)

22.13 Achievements

- **Purpose:** Record participation.
- **Main UI elements:** Achievement and certificate entries.
- **User actions:** View achievements.
- **Expected result:** Participation history is visible.
- [\[Insert Screenshot — Achievements\]](#)

22.14 EventHub AI

- **Purpose:** Help users find event information.
- **Main UI elements:** Chat-style input and responses.
- **User actions:** Ask questions about events.
- **Expected result:** Relevant suggestions (demo / rule-based in the prototype).
- [\[Insert Screenshot — EventHub AI\]](#)

22.15 Host Dashboard

- **Purpose:** Overview for organizers.
- **Main UI elements:** Event, registration and attendance statistics; quick actions.
- **User actions:** Navigate to event management.
- **Expected result:** The host sees the state of their events.
- [\[Insert Screenshot — Host Dashboard\]](#)

22.16 Create Event

- **Purpose:** Create a new event.
- **Main UI elements:** Form for title, description, category, date, time, venue, type, capacity, deadline, eligibility, rules, speakers, schedule, banner and meeting link.
- **User actions:** Fill in details and submit for approval.
- **Expected result:** The event is created with *Pending* verification; validation errors appear for invalid input.
- [\[Insert Screenshot — Create Event\]](#)

22.17 Manage Events

- **Purpose:** Maintain the host's events.
- **Main UI elements:** Event list with status and verification badges, edit and cancel actions.
- **User actions:** Edit, cancel or delete events.
- **Expected result:** Changes are reflected in the list.
- [\[Insert Screenshot — Manage Events\]](#)

22.18 Participants

- **Purpose:** View and manage registrants.
- **Main UI elements:** Participant table, capacity indicator.
- **User actions:** View and manage participants.
- **Expected result:** The host sees who has registered.
- [\[Insert Screenshot — Participants\]](#)

22.19 Attendance

- **Purpose:** Record attendance.
- **Main UI elements:** Participant list with attendance controls.
- **User actions:** Mark attendance.
- **Expected result:** Attendance status is recorded.
- [\[Insert Screenshot — Attendance\]](#)

22.20 Host Analytics

- **Purpose:** Evaluate event performance.
- **Main UI elements:** Registration, attendance and feedback statistics.
- **User actions:** Review statistics.
- **Expected result:** Insight into event success.
- [\[Insert Screenshot — Host Analytics\]](#)

22.21 Admin Dashboard

- **Purpose:** Platform overview.
- **Main UI elements:** User and event statistics, pending approvals.
- **User actions:** Navigate to management tools.
- **Expected result:** The administrator sees platform status.
- [\[Insert Screenshot — Admin Dashboard\]](#)

22.22 Event Approval

- **Purpose:** Verify submitted events.
- **Main UI elements:** Pending events list, approve and reject actions.
- **User actions:** Approve or reject.
- **Expected result:** Verification status updates to Approved or Rejected.
- [\[Insert Screenshot — Event Approval\]](#)

22.23 User Management

- **Purpose:** Manage users and roles.
- **Main UI elements:** User list, role controls.
- **User actions:** View users, manage roles.
- **Expected result:** User records and roles are updated.

- [\[Insert Screenshot — User Management\]](#)

22.24 Category Management

- **Purpose:** Maintain event categories.
- **Main UI elements:** Category list, add/edit controls, status.
- **User actions:** Create or edit categories.
- **Expected result:** Categories are available for events and filters.
- [\[Insert Screenshot — Category Management\]](#)

22.25 Admin Analytics

- **Purpose:** Platform-wide insight.
- **Main UI elements:** Platform statistics and charts.
- **User actions:** Review statistics.
- **Expected result:** The administrator can monitor platform activity.
- [\[Insert Screenshot — Admin Analytics\]](#)

23. Demo Data

- **Current Prototype.** All records below are **demonstration records** used to show the prototype. They do not represent real events, people or colleges.

23.1 Sample Events

No.	Event
1	MongoDB Tech Odyssey
2	AI & ML Workshop
3	Inter-College Hackathon
4	Web Expo 2026
5	Python Data Science Workshop
6	Technical Symposium
7	Startup Pitch Fest
8	Sports Meet
9	Cultural Fest
10	Career Seminar
11	Cybersecurity Workshop
12	Cloud Computing Bootcamp

23.2 Sample Colleges

- Knowledge Institute of Technology (KIOT)
- ABC Engineering College
- XYZ Institute of Technology

23.3 Demo Student

Field	Value
Name	Sivaperumal B
College	Knowledge Institute of Technology
Department	CSBS
Role	Student

23.4 Demo Host and Admin

The prototype supports Host and Admin roles for demonstration. [Insert demo Host and Admin account names/emails used in the prototype, if applicable. Do not include real passwords.]

24. Testing

Honesty note. The status column shows **Not Tested** until a test has actually been performed. Update it to *Passed* or *Failed* only after real testing, and record the date and tester.

24.1 Functional Testing Scope

Registration, login, event search, filters, event details, event registration, cancellation, capacity, waitlist, schedule, conflict detection, notifications, host event creation, admin approval, participant management, attendance, feedback, analytics and role-based navigation.

24.2 Additional Testing

- **UI testing** — layout, components, badges and consistency.
- **Responsive testing** — mobile, tablet and desktop widths.
- **Navigation testing** — links, menus and role-based routes.
- **Form validation** — required fields, formats and error messages.
- **Error handling** — behaviour on invalid input and empty results.

24.3 Test Cases

Test Case ID	Feature	Input	Expected Result	Status
TC01	Registration	Valid name, email, college, department	Account created	Not Tested
TC02	Registration validation	Empty or invalid email	Validation error shown	Not Tested
TC03	Login	Valid credentials	User reaches the dashboard for their role	Not Tested
TC04	Login (invalid)	Wrong credentials	Error message shown	Not Tested
TC05	Event search	Keyword "hackathon"	Relevant events displayed	Not Tested
TC06	Filters	Select category / college / status	Only matching events displayed	Not Tested
TC07	Event details	Open an event	Complete information displayed	Not Tested
TC08	Event registration	Click Register	Event added to My Events; count updated	Not Tested
TC09	Cancellation	Cancel registration	Event removed from My Events; count updated	Not Tested
TC10	Capacity	Register when event is at capacity	Registration blocked; event shows Full	Not Tested
TC11	Waitlist	Join waitlist for a full event	User added to waitlist	Not Tested
TC12	Schedule	Register for an event	Event appears in My Schedule	Not Tested
TC13	Conflict detection	Register for two overlapping events	Conflict warning displayed	Not Tested
TC14	Notifications	Trigger an event update	Notification displayed	Not Tested
TC15	Host event creation	Submit Create Event form	Event created with Pending status	Not Tested
TC16	Admin approval	Approve a pending event	Verification status becomes Approved	Not Tested
TC17	Admin rejection	Reject a pending event	Verification status becomes Rejected	Not Tested
TC18	Participant management	Open Participants	Registered users displayed	Not Tested
TC19	Attendance	Mark attendance	Attendance status recorded	Not Tested
TC20	Feedback	Submit rating and comment	Feedback stored and shown to the host	Not Tested
TC21	Analytics	Open analytics	Statistics displayed	Not Tested
TC22	Role-based navigation	Log in as Student / Host / Admin	Only role-appropriate menus and pages available	Not Tested
TC23	Responsive layout	Resize to mobile width	Layout adapts without breaking	Not Tested
TC24	Empty / error states	Search with no matches	Empty state displayed	Not Tested

- **Future Production testing** will add API testing with Postman, authentication and authorization tests, and database integration tests.

25. Security Considerations

Practice	Description	Status
Password hashing	Store passwords only as bcryptjs hashes.	• Planned
JWT authentication	Authenticate API requests with signed tokens.	• Planned
Role-based authorization	Restrict actions by Student / Host / Admin.	• Planned (prototype has demo role-based navigation only)
Environment variables	Keep JWT secret and database credentials in <code>.env</code> .	• Planned
Input validation	Validate and sanitize all inputs on the server.	• Planned (basic form validation may exist in the prototype UI)
API authorization	Check permissions on every protected endpoint.	• Planned
Secure MongoDB credentials	Use a least-privilege database user and restricted network access.	• Planned
No secrets in GitHub	Add <code>.env</code> to <code>.gitignore</code> ; commit only <code>.env.example</code> .	• Planned
CORS configuration	Allow requests only from <code>CLIENT_URL</code> .	• Planned
Error handling	Return safe error messages without exposing internals.	• Planned

The current prototype uses mock data and demonstration-level authentication, so it should not be used with real personal data.

26. Advantages

- **Centralized event information** in one platform.
- **Reduced missed announcements.**
- **Easier event discovery** through search, filters and categories.
- **Simplified registration** from the event page.
- **Better schedule management** through My Schedule.
- **Conflict detection** for overlapping events.
- **Better organizer management** of participants and capacity.
- **Event verification** through administrator approval.
- **Improved communication** through notifications.
- **Attendance tracking.**
- **Analytics** for hosts and administrators.
- **Scalable architecture** ready for MongoDB Atlas and real authentication.
- **Multi-role platform** serving students, hosts and administrators.

27. Limitations

The current prototype has the following limitations, stated openly:

- It uses **mock data**; nothing is persisted in a real database.
- **Authentication may be demo-based** and is not secure.
- **MongoDB integration is planned**, not implemented.
- **Backend API integration is planned**, not implemented.
- The **EventHub AI assistant uses demo / rule-based logic**.
- **Notifications are prototype-level**; no email or push delivery exists.
- **QR functionality may be demonstration-level**.

These limitations are expected to be addressed during the production integration phase.

28. Scalability and Future Enhancements

- The following are **future** improvements and are **not** part of the current prototype.

No.	Enhancement	Description
1	Real MongoDB integration	Persistent storage in MongoDB Atlas.
2	Real authentication	Secure accounts for all roles.
3	JWT-based sessions	Token-based session management.
4	Email notifications	Event updates by email.
5	Push notifications	Browser or mobile alerts.
6	QR attendance scanning	Attendance via QR scanning.
7	Real AI integration	A genuine AI-powered assistant.
8	Advanced recommendation engine	Smarter personalized suggestions.
9	Calendar integration	Sync events to external calendars.
10	Certificate generation	Automatically generated certificates.
11	Advanced analytics	Deeper reporting and trends.
12	College-level deployment	Deployment within individual colleges.
13	Multi-college network	A shared network of colleges.
14	Mobile application	Native or cross-platform app.
15	Real-time event updates	Live updates without page refresh.
16	Cloud deployment	Hosted production deployment.
17	Automated reminder system	Scheduled reminders for registered events.

The service abstraction layer, MVC backend plan and MongoDB Atlas design allow these to be added incrementally.

29. Conclusion

EventHub transforms fragmented college event communication into a centralized digital ecosystem. Instead of searching WhatsApp groups, posters, emails and separate forms, students use one platform to find and manage events; organizers use one platform to run them; and administrators use one platform to verify and oversee them.

User	Workflow
Students	Discover → Register → Schedule → Attend → Feedback
Organizers	Create → Manage → Track → Analyze
Administrators	Verify → Manage → Monitor

The current prototype demonstrates the complete product experience using a modular React frontend, mock data and a service abstraction layer. Its architecture is designed for future integration with Node.js, Express.js, MongoDB Atlas, JWT authentication and real-time services, so the prototype can grow into a production system without a redesign of the user interface.

"Every College Event. One Place."

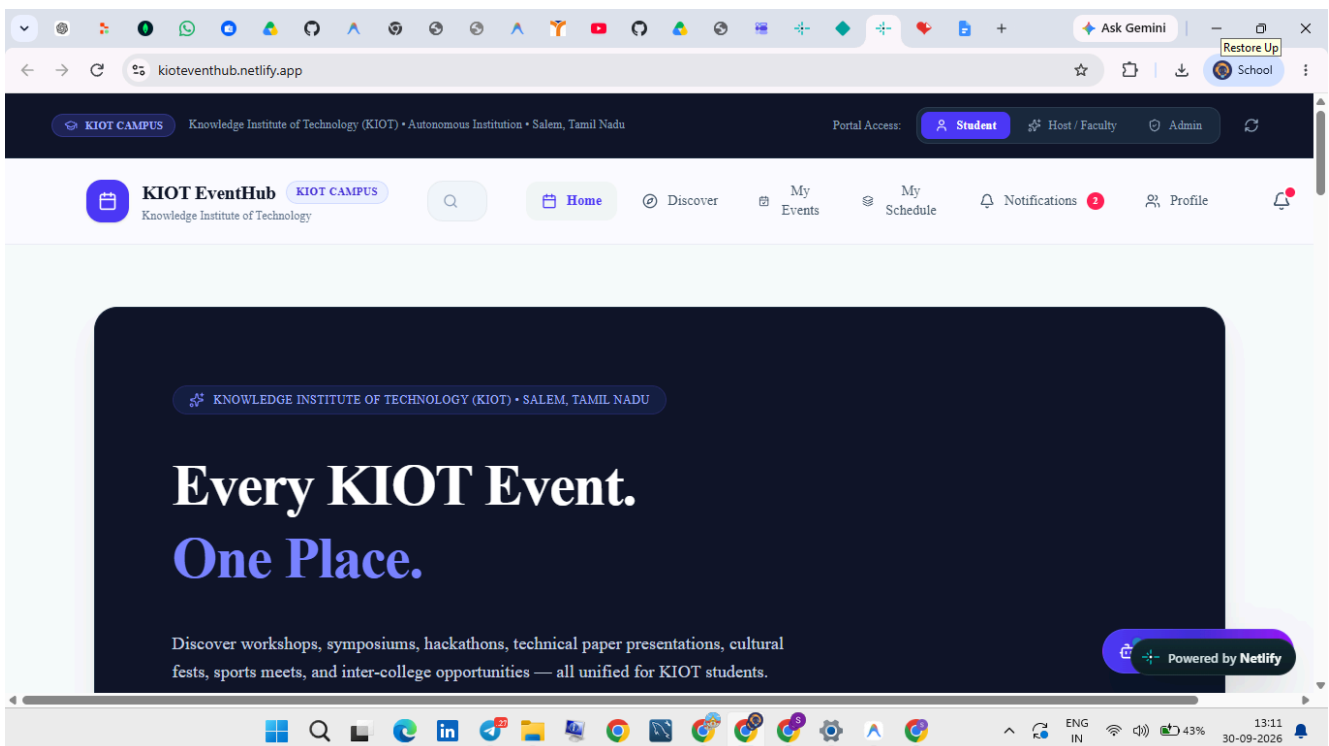
"EventHub — Discover. Register. Schedule. Never Miss an Event."

30. References / Project Links

Item	Link
GitHub Repository	https://github.com/Siva-2406/EventHub-for-KIOT
Live Demo	https://kioteventhub.netlify.app/
Demo Video	https://drive.google.com/drive/folders/1jxMN6v01-4uCJB1V4FSmZGwAll4bLRID

Recommended Diagrams to Insert

- 1 System Architecture
- 2 Technical Architecture
- 3 ER Diagram
- 4 Student User Flow
- 5 Host User Flow
- 6 Admin User Flow
- 7 Complete Event Lifecycle
- 8 MVC Architecture
- 9 Database Collection Structure
- 10 Project Folder Structure



Ask Gemini

Restore Up

School

kioteventhub.netlify.app

KIOT EventHub

Knowledge Institute of Technology

KIOT CAMPUS

Home

Discover

My Events

My Schedule

Notifications

2

Profile

LIVE NOW ON CAMPUS

Session in progress • Free entry for students

MongoDB Tech Odyssey

Deep dive into MongoDB Atlas, Vector Search, and modern document models with live coding.

📍 KIOT Innovation Lab (Hall 3B, 2nd Floor)

👤 128 active attendees

Host: Prof. Ramesh K.

JOIN LIVE SESSION

VIEW EVENT DETAILS →

Powered by Netlify

Recommended For You

Windows

Search

Taskbar icons: Edge, File Explorer, Mail, Calendar, Photos, Settings, etc.

ENG IN

43%

13:11

30-09-2026

Ask Gemini

Restore Up

School

kioteventhub.netlify.app

KIOT EventHub

Knowledge Institute of Technology

KIOT CAMPUS

Home

Discover

My Events

My Schedule

Notifications

2

Profile

Discover Campus & Inter-College Events

Browse through workshops, technical symposiums, hackathons, and cultural fests.

All Colleges

My College (KIOT)

Inter-College

Search KIOT events, workshops, hackathons, symposiums...

All Modes

All Statuses

Sort: By Date

All Categories

Technology

Hackathon

Workshop

Seminar

Symposium

Web Expo

Cultural

Sports

Career

Entrepreneurship

Communi

Showing 8 events at Knowledge Institute of Technology

CAREER

ONLINE

COMPLETED

TECHNOLOGY

OFFLINE

LIVE NOW

AT & Machine Learning Workshop

WORKSHOP

OFFLINE

Powered by Netlify

Reset Filters

Windows

Search

Taskbar icons: Edge, File Explorer, Mail, Calendar, Photos, Settings, etc.

ENG IN

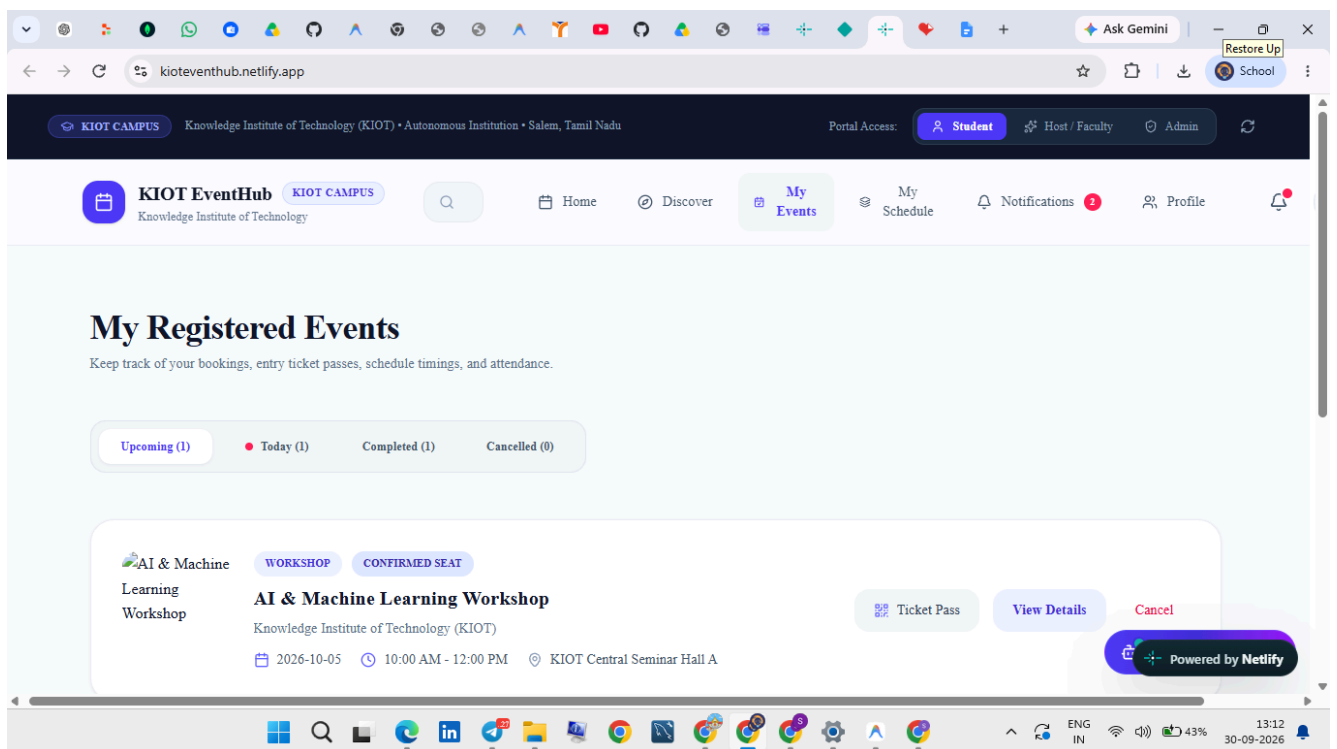
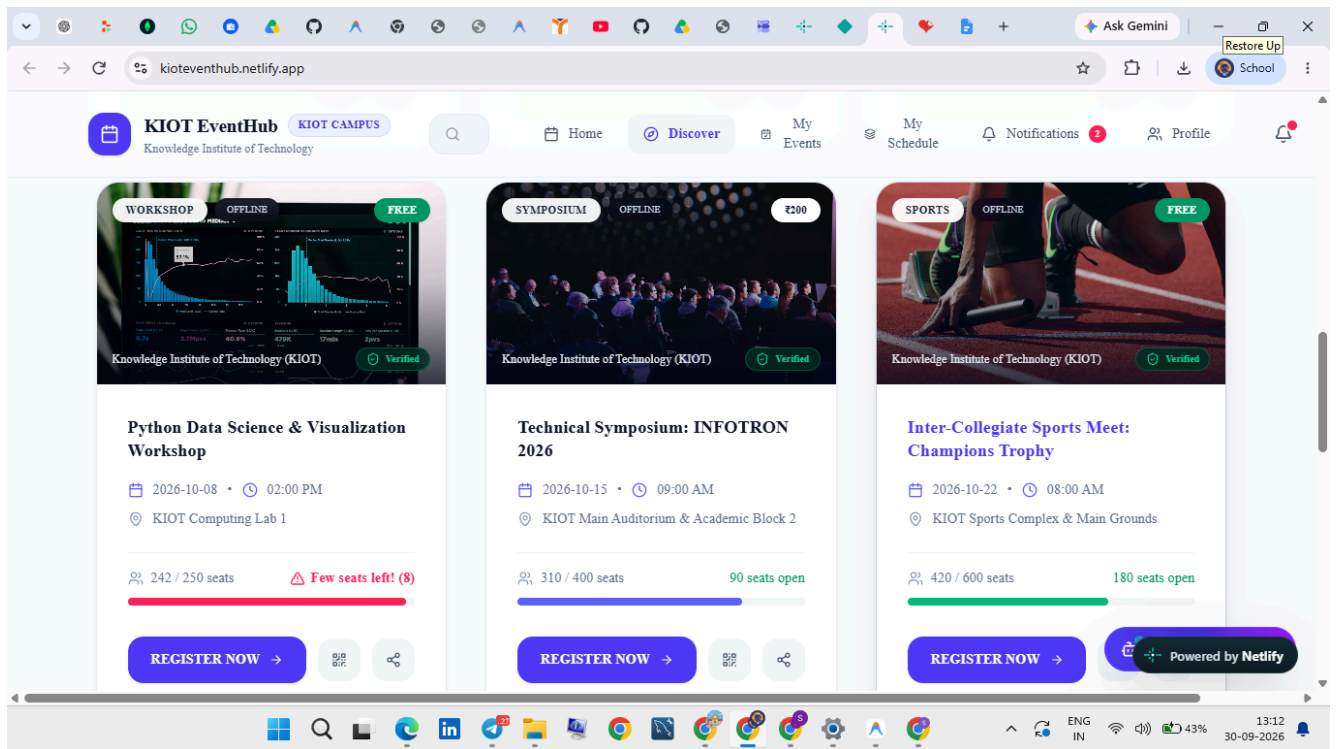
43%

13:12

30-09-2026

EventHub — Every College Event. One Place.

Page 10



Ask Gemini

kioteventhub.netlify.app

Restore Up

School

KIOT CAMPUS

Knowledge Institute of Technology (KIOT) • Autonomous Institution • Salem, Tamil Nadu

Portal Access:

Student

Host / Faculty

Admin

KIOT EventHub

KIOT CAMPUS

Knowledge Institute of Technology

Q

Home

Discover

My Events

My Schedule

Notifications 2

Profile

My Event Schedule Timeline

Chronological calendar of all sessions, workshops, and lab competitions you are registered for.

⚙️ ⚡ Test Schedule Conflict Overlap

📅 Friday, September 25, 2026 (1 event)

06:00 PM - 08:30 PM CAREER

Career & Placement Seminar: Cracking Top Product Firms

📍 Virtual Classroom (Google Meet) • Knowledge Institute of Technology (KIOT)

Details →

📅 Wednesday, September 30, 2026 (1 event)

Powered by Netlify

Windows Taskbar

13:12 30-09-2026

Ask Gemini

kioteventhub.netlify.app

Restore Up

School

KIOT CAMPUS

Knowledge Institute of Technology (KIOT) • Autonomous Institution • Salem, Tamil Nadu

Portal Access:

Student

Host / Faculty

Admin

KIOT EventHub

KIOT CAMPUS

Knowledge Institute of Technology

Q

Home

Discover

My Events

My Schedule

Notifications 2

Profile

Notifications & Smart Reminders

Stay on top of start times, registration deadlines, waitlist openings, and schedule alerts.

🔔 Mark all as read

All (4)

Unread (2)

Reminders

Event Updates

🔔

LIVE NOW: MongoDB Tech Odyssey

MongoDB Tech Odyssey is currently in progress at KIOT Innovation Lab! Click to join session or view live updates.

View Event Details →

10 mins ago

Powered by Netlify

Windows Taskbar

13:12 30-09-2026

kioteventhub.netlify.app

Ask Gemini

Restore Up

School



KIOT EventHub
Knowledge Institute of Technology

KIOT CAMPUS

Home

Discover

My Events

My Schedule

Notifications 3

Profile

My Achievements & Verified Certificates

Click any certificate card below to view, verify, or download your authenticated digital campus credential.



✓ VERIFIED

CAREER EXCELLENCE
Top Product Placement Masterclass
Career & Placement Seminar: Cracking Top Product Firms

September 25, 2026

View PDF



✓ VERIFIED

HACKATHON BADGE
Hackathon Contender: HackKIOT 2026
Annual KIOT Campus Hackathon

August 14, 2026

View PDF



✓ VERIFIED

TECHNICAL WORKSHOP
Full Stack Modern Web Fellow
Full Stack React & Node Workshop Series

July 10, 2026

View PDF

Powered by Netlify

kioteventhub.netlify.app

Ask Gemini

Restore Up

School



KIOT EventHub
Knowledge Institute of Technology

KIOT CAMPUS

Home

Discover

My Events

My Schedule

Notifications 3

Profile



Sivaperumal B Verified Student

Knowledge Institute of Technology (KIOT)
Computer Science and Business Systems (CSBS)
2k25csbs47@kiot.ac.in

Edit Preferences

4
Events Registered

2
Events Attended

3
Certificates Earned

2
Feedback Reviews

INTERESTS & TOPIC PREFERENCES (POWERS SMART RECOMMENDATIONS)

Technology

Hackathons

AI

Workshops

Entrepreneurship

Powered by Netlify

EventHub — Every College Event. One Place.

Page 10